# Spring Boot Annotations Reference

Must-know annotations & efficiency patterns for Java development

TodoApp · Spring Boot 3.x

JPA / ENTITY LOMBOK REST / MVC VALIDATION SPRING CORE EFFICIENCY

## JPA / ENTITY

Map Java classes and fields to database tables and columns

### @Entity

Marks the class as a JPA-managed database entity. Every table model needs this.

```
@Entity
public class Todo { ... }
```

### @Table(name="...")

Sets a custom table name. Optional — defaults to class name if omitted.

```
@Table(name = "todos")
```

### @Id

Declares the primary key field. Every entity must have exactly one.

```
@Id
private Long id;
```

### @GeneratedValue

Auto-generates the primary key. Use IDENTITY for MySQL auto-increment.

```
@GeneratedValue(
  strategy = GenerationType.IDENTITY
)
```

### @Column

Customise column name, nullability, uniqueness, length. Use when defaults aren't enough.

```
@Column(name="task_title",
        nullable=false, length=255)
```

### @Enumerated

Stores an enum. Always use EnumType.STRING — ordinal breaks if you reorder values.

```
@Enumerated(EnumType.STRING)
private Priority priority;
```

### @ManyToOne

Many todos belong to one category. Most common relationship in a Todo app.

```
@ManyToOne
@JoinColumn(name = "category_id")
private Category category;
```

### @OneToMany

One category has many todos. Pair with mappedBy to avoid a duplicate join table.

```
@OneToMany(mappedBy = "category")
private List<Todo> todos;
```

### @ManyToMany

Todos ↔ Tags (many-to-many). Needs @JoinTable on the owning side.

```
@ManyToMany
@JoinTable(name="todo_tags",
  joinColumns=@JoinColumn(name="todo_id"),
  inverseJoinColumns=@JoinColumn(name="tag_id"))
private Set<Tag> tags;
```

### @JoinColumn

Names the foreign key column on the owning side of a relationship.

```
@JoinColumn(name = "category_id")
```

LOMBOK Eliminate boilerplate — getters, setters, constructors, builders

### @Data

Combines @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor. One annotation does it all.

### @NoArgsConstructor

Generates an empty constructor. **Required by JPA** — always add this alongside @AllArgsConstructor.

### @AllArgsConstructor

Generates a constructor with all fields as parameters. Replaces hand-written constructors.

### @Builder

Enables builder pattern. Cleaner object creation when you have many optional fields.

```
Todo.builder()
  .task("Buy milk")
  .completed(false)
  .build();
```

### @Getter / @Setter

Granular control — add only getters or only setters at class or field level.

### @Slf4j

Injects a `log` field. Use instead of manually creating a Logger.

```
log.info("Task created: {}", id);
```

SPRING CORE Bean registration and dependency injection

### @Service

Marks the service layer. Tells Spring to register it as a bean; semantically communicates business logic.

### @Repository

Marks the data layer. Also enables automatic exception translation (e.g. SQL → DataAccessException).

### @Component

Generic bean. Use when the class doesn't clearly belong to service, repository, or controller.

### @Configuration + @Bean

Declare beans programmatically. Use for third-party classes you can't annotate directly.

```
@Configuration
public class AppConfig {
  @Bean
  public ModelMapper modelMapper() { ... }
}
```

REST / MVC Expose HTTP endpoints and bind request data

### @RestController

= @Controller + @ResponseBody. Every method return value is serialized to JSON automatically.

### @RequestMapping

Base path for the controller. All method-level mappings are relative to this.

```
@RequestMapping("/api/todos")
```

### @GetMapping / @PostMapping / @PutMapping / @DeleteMapping

Method-level HTTP verb shortcuts. Use these instead of @RequestMapping(method=...).

```
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

### @PathVariable

Extracts a value from the URL path segment.

```
@GetMapping("/{id}")
public Todo get(@PathVariable Long id)
```

### @RequestBody

Deserializes the JSON request body into a Java object. Use with DTOs, not entities.

```
public Todo create(
  @RequestBody TodoRequest req)
```

### @RequestParam

Binds a query parameter. Set required=false and provide defaultValue for optional params.

```
@RequestParam(required=false,
  defaultValue="false") boolean done
```

### @ResponseStatus

Set HTTP status code on a method or exception class. Cleaner than returning ResponseEntity.

```
@ResponseStatus(HttpStatus.CREATED)
```

### @ControllerAdvice + @ExceptionHandler

Global exception handling. Centralizes error responses — keeps controllers clean.

```
@ControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity<?> handle(...) { }
}
```

VALIDATION Enforce input rules at the API boundary (add spring-boot-starter-validation to pom.xml)

### @Valid

Triggers validation on the annotated method parameter. Put it before @RequestBody.

```
public Todo create(
  @Valid @RequestBody TodoRequest req)
```

### @NotNull / @NotBlank / @NotEmpty

NotNull: not null. NotBlank: not null + not whitespace. NotEmpty: not null + not empty string/collection.

### @Size(min, max)

String length or collection size constraints.

```
@Size(min=1, max=255)
private String task;
```

### @Min / @Max

Numeric bounds. Works on Integer, Long, etc.

```
@Min(1) @Max(5)
private int priority;
```

### @Email

Validates email format. Useful when you add user accounts later.

### @Pattern(regexp)

Validates a string against a regex. Use for custom formats like codes or slugs.

## EFFICIENCY PATTERNS

Annotations and practices that make your app faster and more maintainable

### Use @Transactional on service methods

`@Transactional`

Wraps the method in a DB transaction. Add `readOnly=true` on read methods — it's a performance hint to the DB driver and prevents accidental writes.

### Constructor injection, not @Autowired on fields

`@Autowired (avoid on fields)`

Constructor injection makes dependencies explicit, works without reflection, and is testable without a Spring context. With Lombok, just add `@RequiredArgsConstructor` + make the field `final`.

### Custom queries with @Query

`@Query`

Write JPQL or native SQL directly on a Repository method. Avoids loading the whole entity when you only need part of it.

### Prevent circular JSON references

`@JsonIgnore / @JsonManagedReference`

When Todo has a Category and Category has a List<Todo>, Jackson will infinite-loop serializing them. `@JsonIgnore` on the back-reference breaks the cycle.

### Cache repeated reads

`@Cacheable / @CacheEvict`

Caches the return value on first call; returns the cached result on subsequent calls. `@CacheEvict` clears it when data changes. Add spring-boot-starter-cache.

### Run slow work off the main thread

`@Async`

Makes the method run in a thread pool. Needs `@EnableAsync` on your config class. Return `CompletableFuture<T>` if you need the result later.

## QUICK LOOKUP

Sorted by how often you'll need them in a typical Todo App

| ANNOTATION | WHERE IT GOES | WHY IT MATTERS |
|---|---|---|
| `@Entity` MUST | Model class | Without it, Hibernate ignores the class entirely |
| `@Id + @GeneratedValue` MUST | PK field | Required by JPA — every entity needs a primary key |
| `@NoArgsConstructor` MUST | Entity class | JPA needs to instantiate entities without arguments |
| `@RestController` MUST | Controller class | Without it, methods return view names, not JSON |
| `@Valid` MUST | @RequestBody param | Validation annotations on DTOs do nothing without this |
| `@Transactional` BEST PRACTICE | Service methods | Prevents partial writes; add readOnly=true on reads |
| `@JsonIgnore` BEST PRACTICE | Back-reference field | Stops infinite loops in bidirectional relationships |
| `@Enumerated(STRING)` BEST PRACTICE | Enum fields | ORDINAL breaks silently if enum order changes |
| `@Query` PERFORMANCE | Repository method | Avoids SELECT * when you only need specific columns |
| `@Cacheable` PERFORMANCE | Service read methods | Cuts repeated DB hits for data that rarely changes |